

MEDITACTIVE API

PRESENTAZIONE PROGETTO NODE.JS



Roxana Yanez



The slide features two large, semi-transparent purple circles. One is in the top right corner, and the other is on the left side, partially cut off by the edge of the frame.

Obiettivo del progetto

Realizzare API JSON RESTful per supportare una app di meditazione.

- **Gestire utenti**, obiettivi, intervalli temporali e associazioni tra intervalli e obiettivi.
- **Fornire endpoint** chiari e coerenti con naming, metodi HTTP e status code REST.
- Consegnare un **backend** testabile, documentato e ricostruibile tramite migrations SQL.

User

email, firstName, lastName

Goal

title, description, category

GoalInterval

userId, startDate, endDate

GoalIntervalGoal

intervalId, goalId

Analisi delle entità

- **User**: rappresenta l'utente dell'app, identificato da email, nome e cognome.
- **Goal**: rappresenta un obiettivo predefinito, con titolo, descrizione e categoria.
- **GoalInterval**: rappresenta il periodo in cui un utente lavora sui propri obiettivi.
- **GoalIntervalGoal**: tabella ponte che permette la relazione molti-a-molti tra intervalli e obiettivi.

Scelte tecnologiche

- **Node.js:** ambiente JavaScript lato server, adatto alla creazione di API leggere.
- **Express.js:** framework semplice per gestire routing, middleware e risposte HTTP.
- **MySQL:** database relazionale adatto a gestire relazioni tra entità.
- **mysql2:** driver usato con prepared statement per evitare query vulnerabili a SQL Injection.
- **dotenv:** gestione delle variabili sensibili fuori dal codice sorgente.

Request HTTP

Routes

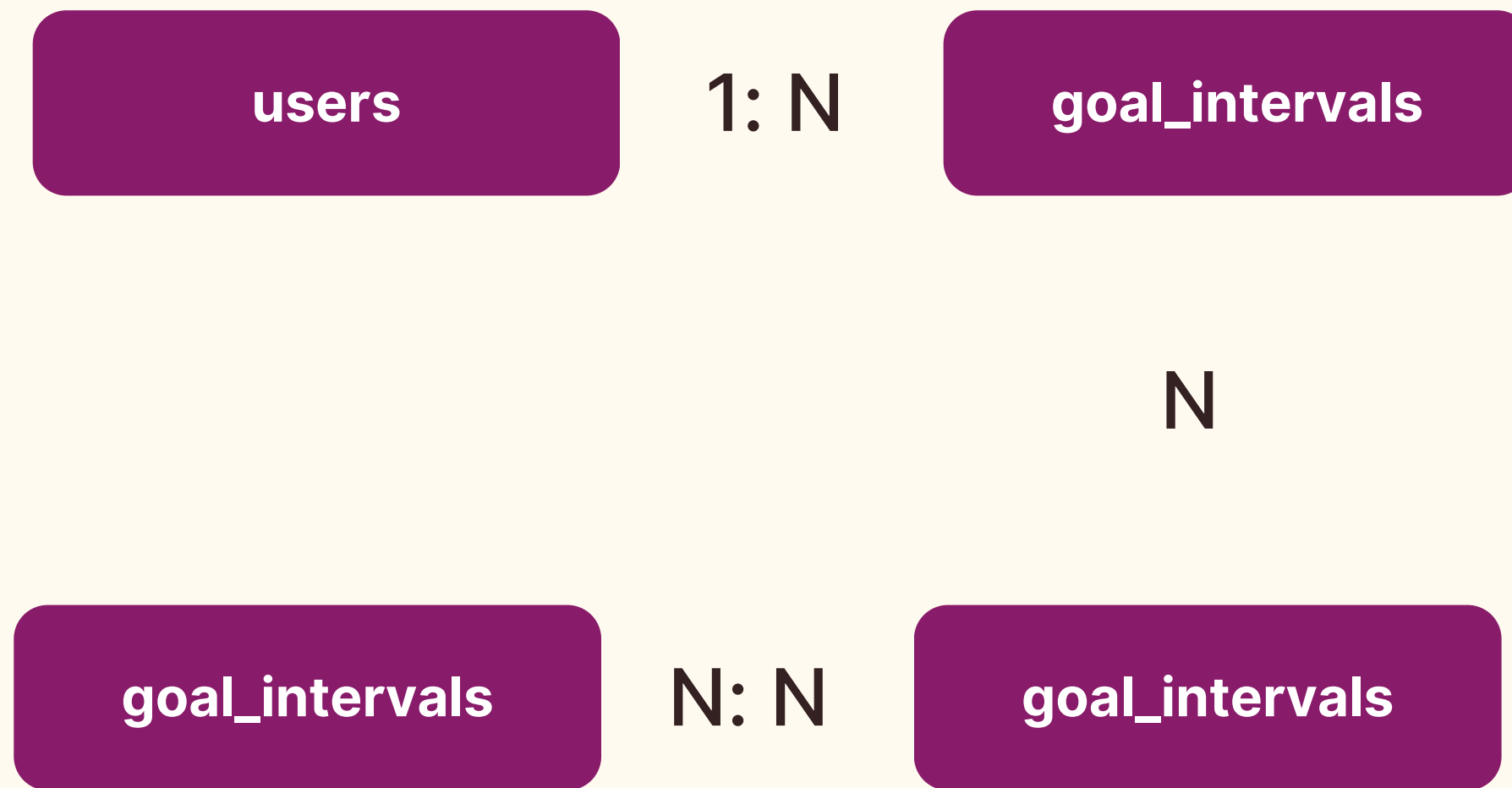
Controllers

Repositories

MySQL

Architettura backend

- Il progetto è stato **diviso in livelli** per separare responsabilità diverse.
- Le **routes** definiscono gli endpoint e collegano URL e metodi HTTP ai controller.
- I **controller** gestiscono input, validazioni, status code e risposte JSON.
- I **repository** contengono solo le query SQL verso MySQL.
- I **middleware** centralizzano errori, rotte non trovate e funzioni asincrone.



Database e relazioni

La relazione multi-a-molti tra goal_intervals e goals viene gestita tramite la tabella ponte goal_interval_goals.

API REST

implementate

Status code

200, 201, 204, 400, 404,
409, 500

Formato risposte

JSON coerente e
leggibile

Filtri

?goalId=1&startDate;=2026-
06-01&endDate;=2026-06-
30

- **Users:** GET, POST, PUT, DELETE su /api/users.
- **Goals:** GET, POST, PUT, DELETE su /api/goals.
- **Goal intervals:** GET, POST, PUT, DELETE su /api/goal-intervals.
- **Associazione obiettivo/intervallo:** POST /api/goal-intervals/:id/goals.
- **Rimozione associazione:** DELETE /api/goal-intervals/:intervalld/goals/:goalld.
- **Filtri** sugli intervalli: goalId, startDate, endDate tramite query string.

Prepared statement

Esempio: WHERE id = ? con valori passati in array.

Validazioni

Email, campi obbligatori, date coerenti, duplicati e associazioni esistenti.

Error handling

Risposte coerenti senza duplicare try/catch in ogni punto.

Sicurezza e gestione errori

- Tutte le **query** usano prepared statement con placeholder, evitando concatenazioni SQL pericolose.
- Le variabili sensibili, come password database, sono lette dal **file .env** escluso da Git.
- **ApiError** permette di gestire in modo chiaro errori controllati come 400, 404 e 409.
- **errorHandler** e **notFoundHandler** centralizzano le risposte di errore in formato JSON.

Testing automatico

- Sono stati aggiunti **test** automatici per verificare le API principali.
- Jest esegue le suite di test.
- Supertest simula richieste HTTP verso l'app Express.
- **Sinon** crea stub dei repository, permettendo di testare i controller senza dipendere dal database reale.
- I test coprono casi positivi, validazioni, risorse non trovate e conflitti.

The background features two large, semi-transparent purple circles. One circle is positioned on the left side, partially cut off by the edge of the frame. The other circle is located in the upper right corner.

Repository GitHub

<https://github.com/Roxy-ya/meditactive-api>

..... **GRAZIE**

